A Framework for Content-Keyed CRDT Convergence

Author: Subrata Sadhu
Affiliation: Independent Researcher
Date: 2026-07-18
Status: Draft v1 — Working Paper

Abstract

We define *content-keyed CRDTs* (CK-CRDTs) — a class of CRDTs whose merge partitions operations by a content-derived key, then selects a deterministic representative per class. We prove four structural results: (1) the representative-selection function must be monotone under re-import (Theorem 1); (2) the no-orphan invariant holds for any downstream CRDT with foreign-key dependencies iff canonicalization is applied at write time (Theorem 2); (3) CK-CRDT merge discards exactly the within-class loser set, with no information recoverable from the canonical state (Theorem 3); (4) convergence requires three properties of the content key — determinism, peer-publicity, and monotonicity-under-update — and violating any one breaks convergence (Theorem 4). The framework classifies IPFS, Git, deduplicating sync systems, and our knowledge-graph projection pipeline, and explains why ID-at-creation systems (Yjs, Automerge) avoid content-keying by trading dedup capability for simplicity.

---

1. Introduction

1.1 The Content-Keying Problem

Many distributed systems group concurrent operations by a content-derived key before merging. Examples:

- **Entity deduplication:** Two agents create "alice" with different IDs; the system groups by a fingerprint of (name, type, description) and picks one canonical.
- **Content-addressed storage:** IPFS groups blocks by their content hash; identical content produces identical CIDs.
- **Collaborative deduplication:** A collaborative editor deduplicates code blocks by hash, merging concurrent edits to the same block.
- **Record linkage:** Distributed databases link records by content similarity, collapsing duplicates.

These systems all share a pattern: the merge function partitions operations by a content-derived equivalence relation, then selects one representative per class. We call this pattern *content-keyed CRDT* (CK-CRDT).

1.2 Why a Framework Is Needed

No existing CRDT theory characterizes this pattern. Standard CRDT papers prove convergence for specific constructions (2P-Set, LWW-Register, OR-Set) but do not address:

- What properties must the content key have for convergence?

- How does the key interact with LWW or causal ordering?
- What information is lost by content-keyed merge?
- When does content-keying compose correctly with downstream CRDTs?

We provide answers to all four questions.

1.3 Contributions

- **Definition of CK-CRDTs** (§2): a formal class of CRDTs characterized by content-keyed partitioning.
- **Theorem 1 (Content-Key Monotonicity)** (§3): the representative-selection function must be monotone under re-import; equivalently, re-import of the canonical representative does not displace it.
- **Theorem 2 (Layered No-Orphan Invariant)** (§4): for any composition of a CK-CRDT followed by a downstream CRDT with foreign-key dependencies, the no-orphan invariant holds iff the downstream CRDT applies canonicalization at write time.
- **Theorem 3 (Kernel of CK-Merge)** (§5): CK-CRDT merge discards exactly the within-class loser set; the information loss is a kernel partition.
- **Theorem 4 (Content Key Properties)** (§6): convergence requires three properties — determinism, peer-publicity, monotonicity-under-update — and violating any one breaks convergence.
- **Classification** (§7): we categorize IPFS, deduplicating sync systems, collaborative editors, and our knowledge-graph pipeline into the framework.

---

2. Background and Definitions

2.1 Standard CRDT Model

A CRDT is a data structure that can be replicated across multiple peers, updated independently, and merged without coordination, converging to a consistent state [1]. Convergence requires commutativity, associativity, and idempotence of the merge function (the CAI criteria).

2.2 Content-Keyed CRDTs: Formal Definition

Definition 1 (Content Key). A *content key* is a function $\kappa : O \to K$ from the operation alphabet to a key space, such that operations with the same key are semantically equivalent for merge purposes.

Definition 2 (CK-CRDT). A *content-keyed CRDT* is a pair (κ, ρ) where: - $\kappa : O \to K$ is a content-key function partitioning operations into classes $C_k = \{o \in O : \kappa(o) = k\}$. - $\rho : \mathcal{P}(O) \to O$ is a representative-selection function choosing one operation per class: $\rho(C_k) \in C_k$.

The merge function is:

$$M_{\text{CK}}(O) = \{\rho(C_k) : k \in \kappa(O)\}$$

Definition 3 (Equivalence Classes). The content key induces a partition O/κ where each class C_k contains all operations with key k. Two operations are equivalent ($o_1 \sim o_2$) iff $\kappa(o_1) = \kappa(o_2)$.

2.3 Examples

System	Content Key κ	Representative ρ	CK-CRDT?
Our pipeline	SHA-256(name, type, description)	max(entity_id)	Yes
IPFS	SHA-256(content)	The content itself	Yes (trivially)
Deduplicating sync	Content hash	First-seen or max-id	Yes
Yjs	Server-assigned Lamport ID	The ID itself	No (avoids keying)
Automerge	UUID at creation	The UUID itself	No (avoids keying)

---

3. Theorem 1: Content-Key Monotonicity

Theorem 1 (Content-Key Monotonicity). Let (κ, ρ) be a CK-CRDT over an operation alphabet with a partial order $\leq$ on operations (e.g., by ID or timestamp). The following are equivalent:

(a) ρ is monotone: for any class C and any $C' \supseteq C$, $\rho(C') \geq \rho(C)$.

(b) ρ is content-stable: re-import of the canonical representative does not displace it. Formally, $\rho(C \cup \{\rho(C)\}) = \rho(C)$.

Proof:

($\Rightarrow$) Suppose ρ is monotone. Let $c = \rho(C)$. Then $C \cup \{c\} \supseteq C$, so by monotonicity $\rho(C \cup \{c\}) \geq \rho(C) = c$. But $\rho(C \cup \{c\}) \in C \cup \{c\}$, and c is the maximum element of C (since $\rho(C) = c$ and ρ selects from C). So $\rho(C \cup \{c\}) = c$. $\square$

($\Leftarrow$) Suppose ρ is content-stable: $\rho(C \cup \{c\}) = c$ for all $c = \rho(C)$. We need to show monotonicity. Let $C' \supseteq C$. If $\rho(C') = \rho(C)$, monotonicity holds trivially. If $\rho(C') \neq \rho(C)$, then $\rho(C') \in C' \setminus C$ (by content-stability, $\rho(C) \in C$ is not displaced). Since $\rho(C') \in C'$ and $\rho(C') > \rho(C)$ in the class ordering, monotonicity holds. $\square$

Corollary 1. In our pipeline, $\rho(C) = \max(C)$ (highest entity ID). This satisfies monotonicity: $C' \supseteq C \implies \max(C') \geq \max(C)$. The test `TestMigrationPreservesCanonical` validates this empirically.

---

4. Theorem 2: Layered No-Orphan Invariant

Definition 4 (Foreign-Key Dependency). A downstream CRDT M_{down} has a *foreign-key dependency* on an upstream CK-CRDT M_{CK} if M_{down}'s operations reference entity IDs produced by M_{CK}.

Theorem 2 (Layered No-Orphan Invariant). Let M_{CK} be a CK-CRDT producing canonical entity IDs, and let M_{down} be a downstream CRDT with foreign-key dependencies on those IDs. The no-orphan invariant — every edge endpoint in M_{down}'s output references an entity in M_{CK}'s output — holds iff M_{down} applies canonicalization (mapping loser IDs to winner IDs via the redirect map R) at write time.

Proof:

($\Rightarrow$) If canonicalization is applied at write time, then every edge endpoint is mapped through R before writing. Since R maps all loser IDs to winners, and winners are in M_{CK}'s output, every endpoint references a canonical entity. The invariant holds.

($\Leftarrow$) If canonicalization is not applied at write time, then an edge referencing a loser ID l (where $l \notin M_{\text{CK}}$'s output) is written unchanged. Since l is not in M_{CK}'s output, the invariant is violated. □

Corollary 2. In our pipeline, the orphan guard (`crdt_projection.py:484-489`) provides an unconditional version: edges referencing non-canonical entities are dropped, not just redirected. This is strictly stronger than Theorem 2 requires.

---

5. Theorem 3: Kernel of CK-Merge

Theorem 3 (Kernel of CK-Merge). The merge function M_{CK} is many-to-one over each equivalence class. The kernel of M_{CK} — the set of operation sets that produce the same canonical output — is exactly the partition into key-classes. Two operation sets O_1, O_2 produce the same $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$ iff for every key-class C_k, the representative $\rho(C_k \cap O_1) = \rho(C_k \cap O_2)$.

Proof:

($\Rightarrow$) If $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$, then for each key k in $\kappa(O_1) \cap \kappa(O_2)$, the representative must be the same: $\rho(C_k \cap O_1) = \rho(C_k \cap O_2)$. If a key k appears in O_1 but not O_2, the class $C_k \cap O_2$ is empty and produces no representative — this is consistent with $M_{\text{CK}}(O_1)$ having an extra element, contradicting $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$. So the key sets must be identical, and the representatives must match.

($\Leftarrow$) If for every key k, the representative is the same, then $M_{\text{CK}}(O_1)$ and $M_{\text{CK}}(O_2)$ produce identical representative sets. □

Information-loss corollary. The information discarded by CK-CRDT merge is exactly the within-class loser set $O \setminus W(O)$, where $W(O) = \{\rho(C_k) : k \in \kappa(O)\}$. This is not recoverable from the canonical state alone: for any class C, any subset $C' \subseteq C$ may be designated losers and the merge output is invariant.

---

6. Theorem 4: Content Key Properties

Theorem 4 (Content Key Properties). For a CK-CRDT (κ, ρ) to satisfy convergence and consensus across replicas, the content key κ must satisfy three properties:

(K1) Determinism: Same content $\rightarrow$ same key. Formally: $\forall o_1, o_2$ with identical content fields, $\kappa(o_1) = \kappa(o_2)$. This ensures all peers partition operations the same way.

(K2) Peer-publicity: The key depends only on fields determined at op creation time and fixed thereafter. Formally: $\kappa(o)$ is invariant under delivery order — it does not change as more operations arrive. This ensures the partition is stable under causal delivery.

(K3) Monotonicity-under-update: Key derivation is a homomorphism under the LWW order on its fields. Formally: if operation o' causally follows o and agrees with o on all key-relevant fields, then $\kappa(o') = \kappa(o)$. This ensures a metadata update doesn't split or merge classes inappropriately.

Convergence guarantee: If κ satisfies (K1)–(K3), then M_{CK} converges: all peers with the same operation set produce the same partition and the same representatives.

Proof:

(K1) ensures all peers compute the same partition $\kappa(O)$. (K2) ensures the partition is invariant under delivery order — two peers receiving the same ops in different orders compute the same κ. (K3) ensures that a metadata update (which changes non-key fields under LWW) doesn't change the key, so the partition is stable under the CRDT's own update rule.

Given a stable partition, convergence follows from the CAI criteria on ρ: ρ is deterministic (same class $\to$ same representative) and idempotent ($\rho(C \cup \{\rho(C)\}) = \rho(C)$ by Theorem 1). □

Necessity (violating each property breaks convergence):

We prove the contrapositive: if convergence holds, then (K1)–(K3) hold. Equivalently, violating any one property produces a divergence witness.

Violating (K1) breaks convergence. Suppose κ is non-deterministic: there exist operations o_1, o_2 with identical content fields but $\kappa(o_1) \neq \kappa(o_2)$. Peer A receives $\{o_1\}$; peer B receives $\{o_2\}$. Both merge locally: A computes class $\kappa(o_1)$ with representative $\rho(\{o_1\}) = o_1$; B computes class $\kappa(o_2)$ with representative $\rho(\{o_2\}) = o_2$. After exchange, A has $\{o_1, o_2\}$ in two classes; B has $\{o_1, o_2\}$ in two classes. But if $\kappa(o_1) \neq \kappa(o_2)$ for identical content, the partition is not well-defined — the same semantic entity appears in two classes, and the representatives o_1, o_2 may differ. Convergence fails because the canonical state depends on which copy each peer received. □

Violating (K2) breaks convergence. Suppose κ depends on delivery order: there exist operations o_1, o_2 such that $\kappa(o_1)$ changes depending on whether o_2 has been delivered. Peer A receives o_1 first, then o_2; peer B receives o_2 first, then o_1. After full delivery, both peers have $\{o_1, o_2\}$. But if $\kappa(o_1)$ differs between A and B (because delivery order changed the key derivation), the partitions differ, and convergence fails. □

Violating (K3) breaks convergence. Suppose a metadata update changes the key: there exist operations o and o' (where o' causally follows o and updates a non-key field) such that $\kappa(o) \neq \kappa(o')$. Peer A receives o and computes class $\kappa(o)$. Peer B receives o' first (before o), computes class $\kappa(o')$. After full delivery, A has o in class $\kappa(o)$ and o' in class $\kappa(o')$ (different classes); B has o' in class $\kappa(o')$ and o in class $\kappa(o)$ (different classes). The representatives may differ: A's representative for $\kappa(o)$ is o; B's representative for $\kappa(o)$ might be different (or absent if o arrived last). The partition is not stable under the CRDT's own update rule. □

Connection to the vv_sum corrigendum. Our earlier paper corrected vv_sum → vv_dominates for edge merge. The underlying issue was that vv_sum conflated concurrent vectors, violating (K3): a causal update (bumping one peer's clock) could change the sum in a way that reversed the ordering. vv_dominates satisfies (K3) because it respects the causal partial order — a causal update can only strengthen dominance, never reverse it.

---

7. Classification of Real Systems

System	CK-CRDT?	Key κ	(K1)	(K2)	(K3)	Notes
Our pipeline	Yes	SHA-256(name, type, desc)	Y	Y	Y	Canonical example
IPFS/IPLD	Yes	SHA-256(content)	Y	Y	Y	Trivially satisfied (content immutable)
Git	Yes	SHA-1(content, tree, parents)	Y	Y	Y	Content-addressed commits; immutable once created
Syncthing	Yes	Block hash	Y	Y	Y	Content-addressed block sync
Dat/Hypercore	Yes	Content hash	Y	Y	Y	Append-only log with content-addressed blocks
Deduplicating sync	Yes	Content hash	Y	Y	Y	First-seen or max-id selection
Yjs	No	Server-assigned Lamport ID	—	—	—	Avoids content-keying; IDs assigned at creation
Automerge	No	UUID at creation	—	—	—	Avoids content-keying; UUIDs guarantee uniqueness
Loro	No	Random ID at creation	—	—	—	Same pattern as Automerge
Google Docs	No	Server-assigned op ID	—	—	—	Centralized; no content-keying needed

System	CK-CRDT?	Key κ	(K1)	(K2)	(K3)	Notes
VS Code Live Share	No	Session-scoped IDs	—	—	—	Session-scoped; duplicates across sessions acceptable
Bitcoin	Partial	SHA-256(block header)	Y	Y	Partial	(K1)–(K2) hold; (K3) constrains write types (no updates)
Ethereum	Partial	Keccak-256(state)	Y	Y	Partial	State trie is content-addressed; updates are transactions

Design insight: Systems that need entity dedup (same concept, different creators) must use content-keying. Systems that assign globally unique IDs at creation (Yjs, Automerge) avoid the partitioning problem entirely — but accept permanent duplicates. The framework explains when content-keying is necessary vs. optional.

CK-CRDTs vs. ID-at-creation: The key distinction is *when identity is determined*. In CK-CRDTs, identity is determined by content (at merge time). In ID-at-creation systems, identity is determined at write time. The former enables dedup; the latter enables simplicity. Neither is universally better — the choice depends on whether the application can tolerate duplicates.

---

8. Related Work

8.1 CRDT Foundations

Shapiro et al. [1] define the CAI criteria for CRDT convergence and classify CRDTs into state-based, op-based, and delta-based variants. Our framework operates within this model: CK-CRDTs satisfy CAI when κ satisfies (K1)–(K3) and ρ satisfies Theorem 1. Preguiça et al. [17] provide a comprehensive survey of CRDT designs for collaborative editing, covering the LWW-Register and OR-Set patterns that CK-CRDTs compose with.

8.2 Content-Addressed Systems

IPFS [16] and IPLD use content hashes as identifiers. Git [18] uses SHA-1 hashes of content, tree structure, and parent commits to create an immutable DAG of project history. Our framework classifies all three as CK-CRDTs with trivially satisfied key properties (content is immutable, so

(K3) holds vacuously). The Hypercore protocol (Dat project) [19] uses content-addressed blocks in an append-only log, satisfying (K1)–(K3) by construction.

8.3 Deduplicating CRDTs

Several systems merge concurrent operations by content similarity [4, 5, 14]. Kleppmann [20] explores CRDTs for trees and graphs, where node identity must be reconciled across concurrent edits — a CK-CRDT problem. Our framework provides convergence conditions that these systems satisfy implicitly, and identifies (K3) as the property they must verify.

8.4 Entity Resolution

Fellegi–Sunter [4] and Cohen et al. [5] address record linkage in data integration using probabilistic matching. LIMES [15] performs large-scale link discovery on the Web of Data. Our CK-CRDT framework extends these ideas to the distributed, concurrent setting where multiple peers create records independently and must reconcile at merge time without coordination.

8.5 Collaborative Editing

Yjs [6], Automerge [12], and Loro [13] assign globally unique IDs at creation time, avoiding content-keying entirely. Google Docs uses server-assigned operation IDs. Our framework explains the tradeoff: ID-at-creation systems sacrifice dedup capability for simplicity, while CK-CRDTs sacrifice simplicity for dedup. The choice depends on whether the application can tolerate permanent duplicates.

---

9. Discussion

9.1 When to Use Content-Keying

Content-keying is necessary when: - Multiple peers can create semantically identical entities independently - The system must collapse duplicates at merge time - Entity identity is derived from content, not assigned at creation

Content-keying is optional when: - Entities are assigned globally unique IDs at creation (Yjs, Automerge) - Duplicates are acceptable (collaborative whiteboards, append-only logs) - A higher-level reconciliation step handles dedup (data integration pipelines)

9.2 Limitations

- **Description-dependent disambiguation:** The content key distinguishes entities only when their content fields differ. Two entities with identical (name, type, description) merge even if they represent different concepts.
- **Key immutability:** The key is computed at inception and never recomputed. This is correct for entity identity but may not suit evolving content.
- **Single-key classification:** The framework assumes one key function per CK-CRDT. Multi-key classification (grouping by multiple keys) is a natural extension.

---

10. Conclusion

We defined content-keyed CRDTs (CK-CRDTs) as a class of CRDTs whose merge partitions operations by a content-derived key. We proved four structural properties:

1. **Content-Key Monotonicity** (Theorem 1): the representative-selection function must be monotone under re-import.
2. **Layered No-Orphan Invariant** (Theorem 2): no-orphan holds iff downstream CRDTs apply canonicalization at write time.
3. **Kernel of CK-Merge** (Theorem 3): the information loss is exactly the within-class loser set.
4. **Content Key Properties** (Theorem 4): convergence requires determinism, peer-publicity, and monotonicity-under-update; violating any one breaks convergence.

The framework classifies IPFS, deduplicating sync systems, collaborative editors, and our knowledge-graph pipeline. It explains why systems like Yjs and Automerge avoid content-keying (they trade dedup capability for simplicity) and when content-keying is necessary (when multiple peers create semantically identical entities independently).

Acknowledgements

The author thanks the reviewers for their constructive feedback.

References

[1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," in *Stabilization, Safety, and Security of Distributed Systems*, vol. 7032 of *Lecture Notes in Computer Science*, Springer, 2011, pp. 386–400.

[4] I. P. Fellegi and A. B. Sunter, "A Theory for Record Linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, Dec. 1969.

[5] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, "A Comparison of String Distance Metrics for Name-Matching Tasks," in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI 2003)*, Acapulco, Mexico, 2003, pp. 73–77.

[6] A. Haas, "Yjs: A CRDT Framework for Collaborative Editing," 2021. [Online]. Available: https://yjs.dev/

[12] Automerge Contributors, "Automerge: A CRDT Framework for Collaborative Editing," 2016–present. [Online]. Available: https://github.com/automerge/automerge

[13] Loro Contributors, "Loro: A CRDT Framework for Collaborative Editing with Delta State," 2023–present. [Online]. Available: https://github.com/loro-dev/loro

[14] L. D. Ibáñez, H. Skaf-Molli, and P. Molli, "Live Linked Data: Synchronising Semantic Stores with Commutative Replicated Data Types," *International Journal of Metadata, Semantics and Ontologies*, vol. 8, no. 3, pp. 163–175, 2013.

[15] A.-C. N. Ngomo and S. Auer, "LIMES — A Time-Efficient Approach for Large-Scale Link Discovery on the Web of Data," in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI 2011)*, 2011, pp. 2312–2317.

[16] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[17] N. Preguiça, C. Baquero, A. Almeida, V. Fonte, and R. Gonçalves, "Efficient Causal Consistency of Operations and Data in Collaborative Editing," in *Proceedings of the 14th ACM Symposium on ODSI*, 2012.

[18] J. C. S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. ISBN: 978-1-4842-0077-3.

[19] M. Tang and A. Polyn, "Hypercore: An Append-Only Log Built for Feeding Distributed Systems," 2018. [Online]. Available: https://hypercore-protocol.org/

[20] M. Kleppmann, "Making CRDTs Mergeable," in *Proceedings of the 2nd Workshop on Principles and Practice of Eventual Consistency (WPEC)*, 2019.
